

CSCE 605 COMPILER DESIGN · MODULE 11 · LECTURE NOTES

Register Allocation

Infinitely many values, sixteen registers.

MODULE OUTCOMES

- Build an interference graph from liveness.
- Explain graph colouring allocation and Chaitin–Briggs.
- Implement linear scan and say when it is the right choice.
- Explain spilling, rematerialisation, and coalescing.
- Explain why SSA makes the problem easier.

1 Allocation as graph colouring

INTERFERENCE, AND THE REDUCTION

Two values *interfere* if there is any point at which both are live — if so, they cannot occupy the same register, because writing one would destroy the other.

Build a graph with a node per value and an edge per interfering pair. Liveness analysis (Module 08 §2) produces exactly the information needed, which is why liveness is the analysis no compiler can do without.

Then assigning registers is colouring that graph with k colours, where k is the number of available physical registers — adjacent nodes must get different colours, exactly as interfering values must get different registers.

Graph colouring is NP-complete, and **Chaitin showed in 1981 that every graph is the interference graph of some program** — so the hardness is genuine and not an artifact of the reduction. There is no special structure in real programs to exploit in general. **Hence heuristics**, and hence the §4 result being so surprising when it arrived.

```

BUILD      interference graph from liveness
COALESCE   merge copy-related nodes if degree does not rise
SIMPLIFY    repeatedly remove a node of degree < k, push on a stack
            -- such a node is ALWAYS colourable, because its
            neighbours use at most k-1 colours between them
SPILL       if every node has degree >= k, pick one by cost and
            push it OPTIMISTICALLY
SELECT      pop the stack, colouring each node differently from
            its neighbours. An optimistically pushed node may STILL
            find a colour -- its high-degree neighbours may share
            colours. If not, really spill it and restart.

```

CHAITIN (1981): spill immediately at degree $\geq k$.

BRIGGS (1989): push optimistically; often a colour exists anyway.

The **degree-below-k argument is the core insight**, and it is worth stating carefully: a node with fewer than k neighbours can always be coloured, whatever happens to the rest of the graph, because its neighbours can collectively use at most $k - 1$ colours and one is therefore left over. **Briggs' optimistic pushing is the difference between the textbook algorithm and the one worth implementing** — a node of degree k or more may still be colourable if several of its neighbours happen to share a colour, and simply trying costs nothing.

2 Spilling and coalescing

CHOOSE WHAT TO SPILL BY COST, NOT BY CONVENIENCE

Spilling a value means storing it to the stack and reloading it at each use, so its cost is the number of definitions and uses, **weighted by loop depth** — a use inside a doubly nested loop is worth a hundred outside one.

The standard metric is cost divided by degree: $(\text{uses} + \text{defs, each weighted by } 10^{\text{loop depth}}) / \text{degree}$.

Spill what is cheap to spill and relieves the most pressure, which is the right shape for the trade.

Rematerialisation is strictly better where it applies. If the value is a constant, or a simple expression of values that are still live, **recompute it at each use rather than reloading it** — no store, no stack slot, and the recomputation is often a single instruction.

A constant is always cheaper to rematerialise than to spill, and an allocator that misses this spills constants to the stack and reloads them — **which is both embarrassing and extremely common in hand-written allocators**. Check for it explicitly.

COALESCING REMOVES THE COPIES SSA DESTRUCTION CREATED

SSA destruction inserts copies at every ϕ (Module 07 §4), and copy propagation and instruction selection leave more. A function can easily have more copies than useful instructions after destruction.

If a copy's source and destination do not interfere, merge their nodes — they get the same register and the copy is deleted entirely. This is the main reason naive ϕ destruction is acceptable.

But merging two nodes produces a node whose degree is the union of theirs, which can push the graph over the colourability threshold and force a spill. **A spill costs far more than the copy it was meant to save** — a register-to-register move is a cycle or less on a modern machine, while a spill is a store, a reload, and possibly a cache miss.

So coalesce conservatively. Briggs' rule merges only if the resulting node has fewer than k neighbours of significant degree; **George's rule** is a different sufficient condition and the two are usually applied together. **Aggressive coalescing — merging whenever possible — was the original approach and it was measurably worse**, which is a good small example of a locally obvious optimisation losing globally.

3 Linear scan

	Graph colouring	Linear scan
Model	An interference graph over values.	Live intervals over a linearised instruction order.
Cost	Can be $O(n^2)$ or worse — graph construction alone is quadratic in the worst case, and spilling restarts the whole process.	$O(n \log n)$, dominated by sorting the intervals.
Quality	Better — typically 5 to 10% fewer spills, more on register-starved targets.	Good enough for the great majority of code.
Used by	GCC and LLVM at -O2 and above.	JIT compilers: HotSpot's C1, V8's baseline tier, and most first-tier JITs.
Incremental	No — a change requires rebuilding the graph.	Easier to update.
When to use it	When compile time is affordable.	When compile time is on the clock — see below.

LINEAR SCAN: INTERVALS ON A LINE

Linearise the basic blocks into a single instruction order, then give each value a live *interval* running from its first definition to its last use in that order.

Sweep through the intervals in order of start position, maintaining the *active set* of intervals that overlap the current point. At each new interval, first expire every active interval that has already ended and free its register.

Then assign a free register if one exists; otherwise spill — conventionally the interval among the active set and the new one that ends *last*, since it is occupying a register for the longest.

The approximation is that an interval is contiguous, which over-states liveness across holes where the value is dead in the middle of its range — and those holes are common with loops and branches. **Interval splitting recovers most of the lost quality** by allowing a value to occupy different registers, or memory, in different parts of its range, and **that is what production JITs actually implement** rather than the textbook algorithm. **A JIT compiles while the user waits**, so an allocator taking ten times longer to remove 8% of spills is simply the wrong trade.

4 The SSA result

SSA INTERFERENCE GRAPHS ARE CHORDAL

A **chordal graph** is one in which every cycle of four or more vertices has a **chord** — equivalently, it has no induced cycle longer than a triangle. **Chordal graphs can be optimally coloured in polynomial time**, by a simple greedy algorithm over a perfect elimination ordering.

The interference graph of a program in SSA form is always chordal. The reason is structural: in SSA, each value's live range is a *subtree* of the dominator tree, and the intersection graph of a family of subtrees of a tree is always chordal.

So the colouring problem that Chaitin proved NP-complete becomes tractable — and better, the minimum number of registers required equals the maximum number of values simultaneously live at any program point, which is a quantity you can simply compute.

Chaitin's hardness result is not contradicted. It says every graph arises from *some* program; SSA programs simply do not produce the hard graphs. **The representation changed the complexity of the problem** — which is Module 07 §2's claim in its strongest possible form, and a genuinely striking result: changing how you write the program down moved a problem from NP-complete to polynomial.

- **Colouring becomes easy; *spilling* does not.** Deciding which values to spill when the pressure exceeds k is still NP-hard, and **spilling is where essentially all the cost is**. The easy part got easier and the expensive part did not.
- **Destruction still inserts copies** (Module 07 §4), and coalescing them optimally is still hard — so the copies that SSA created have to be removed by the machinery of §2 regardless.
- **Register classes and instruction constraints complicate it.** Some instructions demand specific registers (x86 shifts want the count in `cl`; division uses `rdx:rax`), and some values must be in a floating-point class rather than an integer one. Neither fits the clean colouring model.
- **And callee-saved registers, ABI requirements, and precoloured nodes** (Module 10 §3) all break the theory — an argument arriving in `rdi` is a node whose colour was decided before allocation began.
- **So SSA-based register allocation is a real improvement rather than a solution**, and it is used in production (LLVM and others exploit it) without having ended the subject. **A good result that does not solve the whole problem is the normal shape of progress**, and it is worth saying plainly rather than overselling the theorem.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Chaitin — Register allocation and spilling via graph colouring; Briggs et al. — Improvements to graph colouring (free)	The two papers of §1. Briggs' optimistic colouring is the change worth understanding.

Resource	What it covers for this module
Poletto & Sarkar — Linear Scan Register Allocation (free)	The §3 algorithm , from its authors. Short and clear.
Hack, Grund & Goos — Register Allocation for Programs in SSA Form (free)	The chordality result of §4 , with the subtree argument.
Wimmer & Franz — Linear Scan Register Allocation on SSA Form (free)	Interval splitting and the production JIT version of §3.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Build an interference graph from your liveness analysis and render it.
- 2 **Implement Chaitin–Briggs** with simplify, optimistic spill, and select.
- 3 **Disable optimistic pushing** (spill immediately at degree k) and measure the extra spills.
- 4 Implement the spill cost heuristic with loop-depth weighting.
- 5 **Add rematerialisation for constants** and measure how many spills it removes.
- 6 Implement conservative coalescing. Then try aggressive coalescing and measure the resulting spills.
- 7 Implement linear scan and compare spill counts and allocation time against graph colouring on the same functions.
- 8 **Add interval splitting** and measure how much of the gap it closes.
- 9 **Verify empirically that your SSA interference graphs are chordal** by testing for a perfect elimination ordering.
- 10 Reduce k from 16 to 4 and plot spills against k for both allocators.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define interference and state the colouring reduction.
- 2 Why is a node of degree below k always colourable?
- 3 What did Briggs change, and why does it help?
- 4 Give the spill cost heuristic and say when rematerialisation is better.
- 5 Why must coalescing be conservative?
- 6 Compare graph colouring and linear scan on six axes.

- 7 What approximation does linear scan make, and what recovers it?
- 8 Why are SSA interference graphs chordal, and what follows?
- 9 Why did that result not end the subject?